TUNKU ABDUL RAHMAN UNIVERSITY OF MANAGEMENT AND TECHNOLOGY

BMCS2074

ARTIFICIAL INTELLIGENCE

202605 Session, Year 2026/27

Assignment Documentation

Project Title: Fruit and Vegetable Object Detection Using Deep Learning
Programme: RDSY2S1
Tutorial Group: 5
Tutor: Dr. Cheng Kam Ching

Team members

No	Student Name	Student ID	Module In Charge	Signature and Date
1	Tang Rui Jie	2612542	RetinaNet	
2	Chong Zhen Yang	2612484	Mobilnet V2	
3	Chew Jin Yang	2612479	YOLO	

1. Introduction

1.1 Background

Artificial Intelligence (AI) is an important field of computer science that enables computers to perform tasks that normally require human intelligence, such as recognition, analysis, and decision-making. With the development of Deep Learning, AI has been widely applied to image processing and computer vision. Computer vision enables computers to extract visual information from images and videos and analyse their contents.

Object detection is an important task in computer vision. Unlike image classification, which only determines the class of an entire image, object detection can identify the categories of objects and determine their locations within an image. Therefore, when multiple objects are present in an image, an object detection model can identify each object separately and locate it using bounding boxes.

Fruit and vegetable detection is a practical application of object detection technology. Automatically identifying different types of fruits and vegetables can be applied in areas such as agriculture, food management, retail, inventory management, and automated sorting. With the increasing availability of image data and the development of Deep Learning techniques, AI-based analysis of fruit and vegetable images can provide a more automated approach for these applications.

This project focuses on fruit and vegetable object detection by applying Deep Learning-based object detection techniques to identify and localise different fruits and vegetables in images. The performance of different models is evaluated through experiments to investigate their effectiveness for this task.

1.2 Problem Statement

Automated fruit and vegetable recognition and detection present challenges in terms of accuracy and efficiency. Traditional manual identification requires people to inspect fruits and vegetables individually, which can be time-consuming when processing a large number of images or multiple categories. Manual inspection may also be affected by human judgement and errors. In addition, different types of fruits and vegetables may have similar colours, shapes, and textures, while objects in images may vary in size, position, and lighting conditions, making accurate automated detection more challenging.

Although existing deep learning-based object detection techniques can be used to automatically identify and localise objects, different models may perform differently in terms of detection accuracy, localisation ability, and their ability to handle objects of different sizes. Therefore, the performance of a single model may not provide sufficient evidence to determine which approach is more suitable for fruit and vegetable detection.

Therefore, this project investigates the performance of different deep learning-based object detection models for fruit and vegetable detection. Three object detection models will be trained, tested, and compared using the same dataset and appropriate evaluation metrics. The study will investigate differences in classification, object localisation, and overall detection performance, while also analysing the strengths and limitations of each model.

1.3 Objectives/Aims

The main aim of this project is to investigate and develop a Deep Learning-based object detection system for fruits and vegetables. The specific objectives are:

1. **To develop an AI-based object detection system that can automatically identify and localise fruits and vegetables.**
   The system will identify different fruit and vegetable categories in images and locate the detected objects using bounding boxes.
2. **To implement three different Deep Learning-based object detection models.**
   Different AI models will be applied to the same fruit and vegetable detection task to investigate their detection capabilities.
3. **To evaluate the performance of the three models using appropriate evaluation metrics.**
   Metrics such as Precision, Recall, F1-score, and Intersection over Union (IoU) will be used to evaluate the models' classification and localisation performance.
4. **To compare and analyse the detection performance of the three models.**
   The experimental results will be used to identify the strengths, limitations, and overall performance of each model and determine which approach is more suitable for the fruit and vegetable object detection task.

1.4 Significance / Contribution of the Study

This project provides both theoretical and practical value. From a theoretical perspective, the project improves the understanding of different Deep Learning-based object detection approaches by comparing their performance in a multi-class fruit and vegetable detection task. The experimental results provide a basis for comparing different models in terms of classification, object localisation, and overall detection performance.

From a practical perspective, automated fruit and vegetable detection can reduce the reliance on manual identification and inspection and has potential applications in areas such as agriculture, retail, inventory management, food classification, and automated visual inspection. Automatically identifying and localising fruits and vegetables in images can improve the efficiency of visual analysis processes.

In addition, this project provides a comparative evaluation of three different Deep Learning models using the same dataset and appropriate evaluation metrics. The findings can provide useful guidance when selecting an appropriate model for fruit and vegetable detection and help identify the strengths and limitations of different approaches. The results may also provide a foundation for future improvements to automated fruit and vegetable detection systems.

2. Related Work

2.1 Review of previous studies

2.1.1 RetinaNet

RetinaNet is a one-stage object detection algorithm introduced by Lin et al. (2017) to address the class imbalance problem commonly found in dense object detection. In their study, the authors showed that RetinaNet could achieve competitive detection performance while keeping the efficiency advantage of a one-stage detector (Lin et al., 2017).

In this project, the RetinaNet model is based on the RetinaNet ResNet-50 FPN V2 implementation. The model uses a ResNet-50 backbone with a Feature Pyramid Network (FPN) to detect objects at different scales. This is useful for the fruit and vegetable dataset because objects in the images can have very different sizes, from relatively large objects to small objects.

Previous studies have also applied RetinaNet to agricultural and fruit detection tasks (Kirk et al., 2020). These studies suggest that RetinaNet can be used for images containing multiple objects with different sizes and complex backgrounds (Kirk et al., 2020). This is relevant to the current project because the LVIS Fruits & Vegetables dataset contains many objects in a single image, making the detection task more challenging.

However, object size and scene complexity can still affect detection performance. Small objects provide less visual information for the detector, while overlapping objects can make it harder to separate one object from another. These problems are especially relevant to fruit and vegetable detection, where objects can be close together or partially hidden by other objects (Kalantar et al., 2020; Kirk et al., 2020).

Compared with other commonly used object detection methods such as YOLO and Faster R-CNN, RetinaNet provides a different balance between detection accuracy and speed. YOLO methods are generally designed for fast end-to-end detection, while Faster R-CNN uses a two-stage detection process. The choice of detector therefore depends on the dataset and the requirements of the application (Ali et al., 2025).

Overall, previous studies show that RetinaNet is suitable for multi-class and multi-scale object detection, but small and overlapping objects remain difficult cases. These findings provide a useful basis for evaluating the RetinaNet V2 model used in this project on the LVIS Fruits & Vegetables dataset.

2.1.2 MobileNet V2

MobileNetV2 is a lightweight convolutional neural network architecture introduced by Sandler et al. (2018) to reduce the computational requirements of computer vision applications. The model uses depthwise separable convolution, inverted residual blocks, and linear bottlenecks to reduce the number of parameters and computational operations while maintaining effective feature extraction. MobileNetV2 can also be integrated with lightweight detection frameworks such as SSDLite, allowing it to perform object detection with relatively low computational cost. Sandler et al. (2018) demonstrated that MobileNetV2 provides a favourable balance between predictive performance, model size, and computational efficiency, making it suitable for mobile and resource-constrained environments.

Subsequent research has applied MobileNetV2 to agricultural and fruit detection tasks. Zhou et al. (2020) combined MobileNetV2 with SSD to develop a real-time kiwifruit detection system for Android smartphones. The MobileNetV2-based model achieved a true detected rate of 90.8% while maintaining a relatively small model size of 17.5 MB. The study demonstrated that MobileNetV2 can provide effective object detection while requiring fewer computational resources than heavier backbone networks. However, the study focused only on kiwifruit detection, and its performance may vary when applied to multi-class datasets containing fruits and vegetables with similar colours, shapes, and textures.

Compared with heavier detection approaches, MobileNetV2 mainly emphasizes computational efficiency and lightweight deployment. RetinaNet uses Feature Pyramid Networks and Focal Loss to improve multi-scale detection and address foreground-background class imbalance, while YOLO focuses on fast end-to-end object detection. In comparison, MobileNetV2 is commonly used as a lightweight backbone together with detection frameworks such as SSD or SSDLite. Therefore, the different approaches involve trade-offs between detection accuracy, model complexity, inference speed, and computational requirements.

Overall, previous research suggests that MobileNetV2 is suitable for object detection applications where computational efficiency and model size are important considerations. However, its lightweight design may involve trade-offs in detection capability when dealing with complex scenes or visually similar object categories. These characteristics are particularly relevant to fruit and vegetable detection. Therefore, evaluating a MobileNetV2-based approach alongside other detection models can provide a better understanding of its effectiveness in a multi-class fruit and vegetable detection environment.

2.1.3 YOLO

You Only Look Once (YOLO) is a one stage object detection algorithm that performs object localisation and classification within a unified network. Modern YOLO models use multi-scale feature extraction and prediction to detect objects of varying sizes while maintaining efficient inference. YOLO has been widely applied in agriculture because of its balance between detection accuracy, speed, and model size, applicable to embedded devices and edge computing projects (Badgujar et al., 2024).

The model used in this project is YOLO11m. YOLO11 introduces the C3k2 and C2PSA blocks, improving the accuracy to compute trade-off and small object learning while maintaining the balance between accuracy and inference speed. The model family also supports different scales for balancing computational cost and detection performance (Jocher et al., 2025).

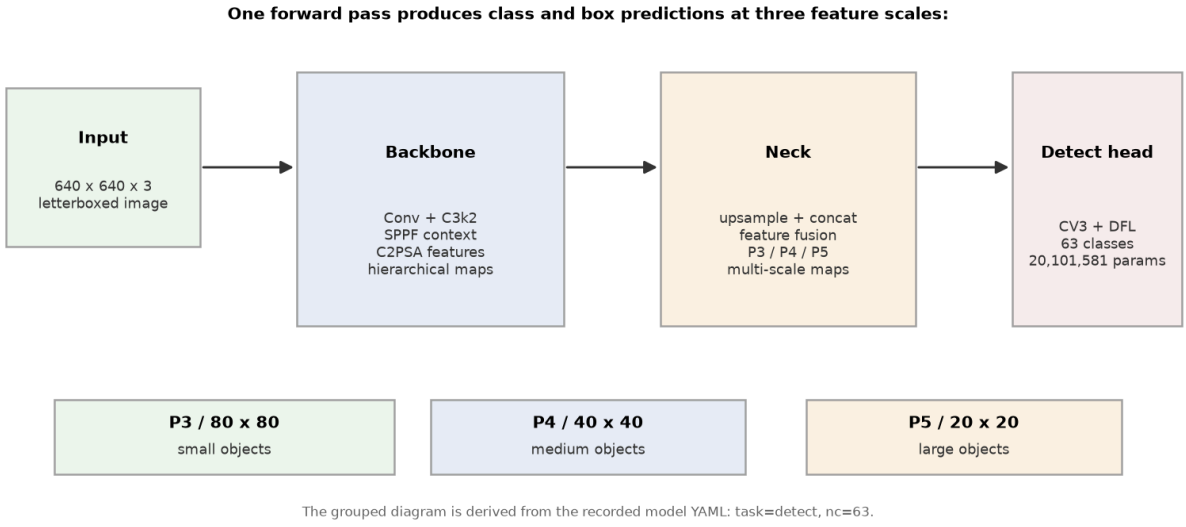


Figure 2.1: YOLO11m grouped architecture and multi-scale prediction path

The multi-scale detection capability of YOLO is particularly relevant to fruit detection because fruits may appear at different sizes and may overlap or become partially occluded. Recent agricultural studies continue to modify YOLO specifically to improve small and occluded fruit detection. For example, Zeng et al. (2025) introduced additional multi-scale feature fusion and a small-object detection layer for citrus detection, achieving an average precision of 92.5%.

Despite its advantages, YOLO performance can still be affected by small objects, occlusions, visually similar objects, and complex backgrounds. A recent review of YOLO-based fruit detection identified small-fruit detection, severe occlusion, and the balance between accuracy and inference speed as continuing challenges (Pugazhendi et al., 2026).

2.1.4 Comparison of Existing Object Detection Methods

The reviewed studies show that RetinaNet, MobileNetV2-based detection approaches and YOLO are different approaches to solve object detection issues. All three methods can be used for fruit and vegetable detection, but they differ in their architecture design, goals and computational requirements. RetinaNet and YOLO are one stage object detection methods that can perform object classification and localization without the use of a separate region proposal stage. By contrast, MobileNetV2 is a lightweight convolutional neural network architecture designed to be used as a backbone for computationally efficient object detection with detection frameworks like SSD or SSDLite.

Method	Main Approach	Key Strength	Main Limitation
RetinaNet	One-stage detector with FPN and Focal Loss	Multi-scale detection and handling foreground-background imbalance	Small and overlapping objects can remain challenging
MobileNet V2	Lightweight CNN backbone with SSD	Low computational cost and small model size	May have limitations in complex and visually similar scenes
YOLO	One-stage end-to-end detection	Fast inference and efficient detection	Small objects and occlusion can affect performance

RetinaNet and YOLO are both one-stage object detection methods, but they focus on different areas. RetinaNet uses Focal Loss to reduce the effect of easy background samples and FPN to detect objects at different scales. YOLO more focuses on fast detection and can process images quickly, which makes it suitable for real-time applications. Some studies on fruit detection have also improved YOLO by adding better feature extraction and small-object detection methods. Therefore, although both RetinaNet and YOLO are one-stage detectors, they use different methods to improve object detection performance.

MobileNetV2-based detection focuses more on reducing the model size and the amount of computation required. For example, Zhou et al. (2020) combined MobileNetV2 with SSD for kiwifruit detection. Their model achieved a true detected rate of 90.8% and had a model size of only 17.5 MB. This shows that MobileNetV2 can be useful for applications where computing resources are limited, such as mobile devices. However, a lightweight model may have more difficulty when dealing with complex images, many object categories, or objects that look similar. Compared with MobileNetV2-based detection, RetinaNet and YOLO are more focused on object detection performance, although they may require more computing resources.

Overall, the previous studies show that each method has its own strengths and weaknesses. RetinaNet is suitable for detecting objects at different scales, YOLO is more suitable when fast detection is important, while MobileNetV2-based detection is useful when a smaller and lighter model is needed. This shows that there is a trade-off between detection performance, detection speed, and computational requirements. Comparing these three methods using the same multi-class fruit and vegetable dataset can help determine which method performs better for this type of detection task.

2.2 Research gap and justification for the current study

2.2.1 RetinaNet

Previous studies have shown that RetinaNet is an effective one-stage object detection model for dense detection tasks. RetinaNet addresses the foreground–background class imbalance problem through the use of Focal Loss, while its Feature Pyramid Network (FPN) enables feature extraction at multiple spatial scales. In this study, RetinaNet is implemented using a **ResNet-50 FPN v2 backbone**, which combines a ResNet-50 convolutional backbone with an FPN-based multi-scale feature representation. These characteristics make RetinaNet a suitable candidate for fruit and vegetable detection. However, its detection performance can still be affected by small objects, object overlap, and partial occlusion, particularly in dense detection environments.

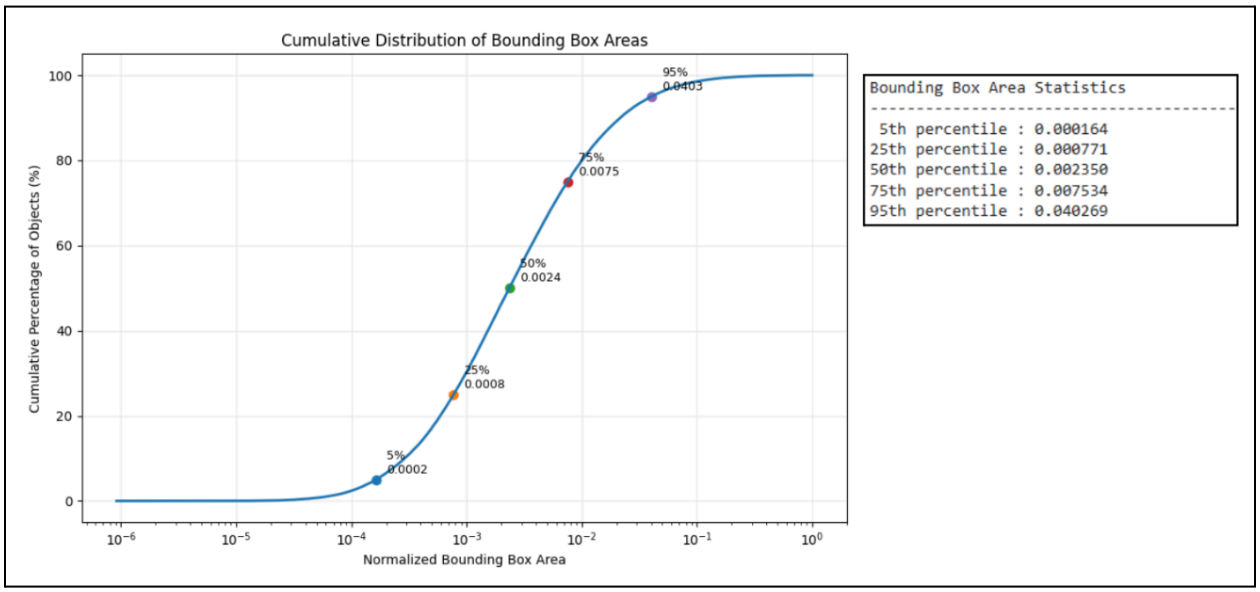


Figure 2.2: Cumulative distribution of bounding box areas in the training dataset

Small objects are particularly challenging for object detection because they contain less visual information and occupy fewer pixels in an image. Detection can become even more difficult when multiple fruits or vegetables overlap with one another or when objects are partially occluded. These challenges are especially relevant to the dataset used in this study because the annotated objects have substantially different sizes. Figure 2.2 shows the cumulative distribution of bounding box areas in the training dataset. The results indicate that **50% of the annotated objects have a normalized bounding box area below 0.00235, while 75% are below 0.00753**. This indicates that a large proportion of the annotated objects occupy relatively small areas within the images, highlighting the importance of evaluating the detector under small-object detection conditions.

Another limitation of previous agricultural object detection studies is that many focus on a limited number of fruit categories or specific types of fruit under particular environmental conditions. For example, some studies primarily investigate individual fruit types such as apples, melons, or kiwifruits. Although these studies demonstrate the applicability of object detection methods to agricultural tasks, their findings may not fully represent a dataset containing a large number of fruit and vegetable categories with different visual characteristics, object sizes, and levels of overlap. Furthermore, differences in datasets, annotation formats, training configurations, and evaluation settings can make direct comparisons between different detection approaches difficult.

Therefore, a research gap remains in evaluating **RetinaNet with a ResNet-50 FPN v2 backbone** under a diverse and complex multi-class fruit and vegetable detection setting. In this study, the model is evaluated using a **63-class fruit and vegetable dataset** containing objects with different sizes, appearances, and levels of overlap. This provides an opportunity to examine whether RetinaNet can maintain effective detection performance under these challenging conditions. Its performance is also compared with YOLO and a MobileNetV2-based detection approach using a consistent evaluation framework, allowing the strengths and limitations of the different approaches to be examined more systematically.

2.2.2 MobileNet V2

Previous studies show that MobileNetV2 is suitable for object detection applications where computational efficiency and small model size are important. Zhou et al. (2020) demonstrated that MobileNetV2 can be combined with SSD for real-time kiwifruit detection on mobile devices while maintaining a relatively small model size and good detection performance. This shows that MobileNetV2 is useful for resource-constrained applications such as mobile and embedded vision systems.

However, most previous MobileNetV2-based fruit detection studies focus on a limited number of fruit categories or specific agricultural environments. For example, Zhou et al. (2020) focused only on kiwifruit detection. Therefore, the performance of MobileNetV2 in a large multi-class fruit and vegetable dataset is still less clear. This is particularly relevant because the dataset used in this study contains 63 classes with fruits and vegetables that may have similar colours, shapes, and textures. A lightweight model may also face greater difficulty when distinguishing visually similar categories or detecting objects in complex scenes.

Therefore, this study evaluates a MobileNetV2-based detection approach using the same 63-class fruit and vegetable dataset as RetinaNet and YOLO. This allows the performance of MobileNetV2 to be examined under a more complex multi-class detection environment and provides a fairer comparison of detection performance, computational efficiency, and model complexity across the three approaches.

2.2.3 YOLO

Although YOLO has been widely applied to fruit detection, much of the existing research focuses on specific crops and custom datasets. Pugazhendi et al. (2026) found out that approximately 95% of reviewed YOLO fruit-detection studies used custom datasets, which limits reproducibility and makes results from different studies difficult to compare directly. Small fruits and severe occlusions also remain common detection challenges.

Another challenge is the sampling distribution used for training. In a large multi-class dataset, some categories may contain substantially more examples than others, which can affect how the model learns rare classes. The dataset used in this study also exhibits this pattern, as shown below:

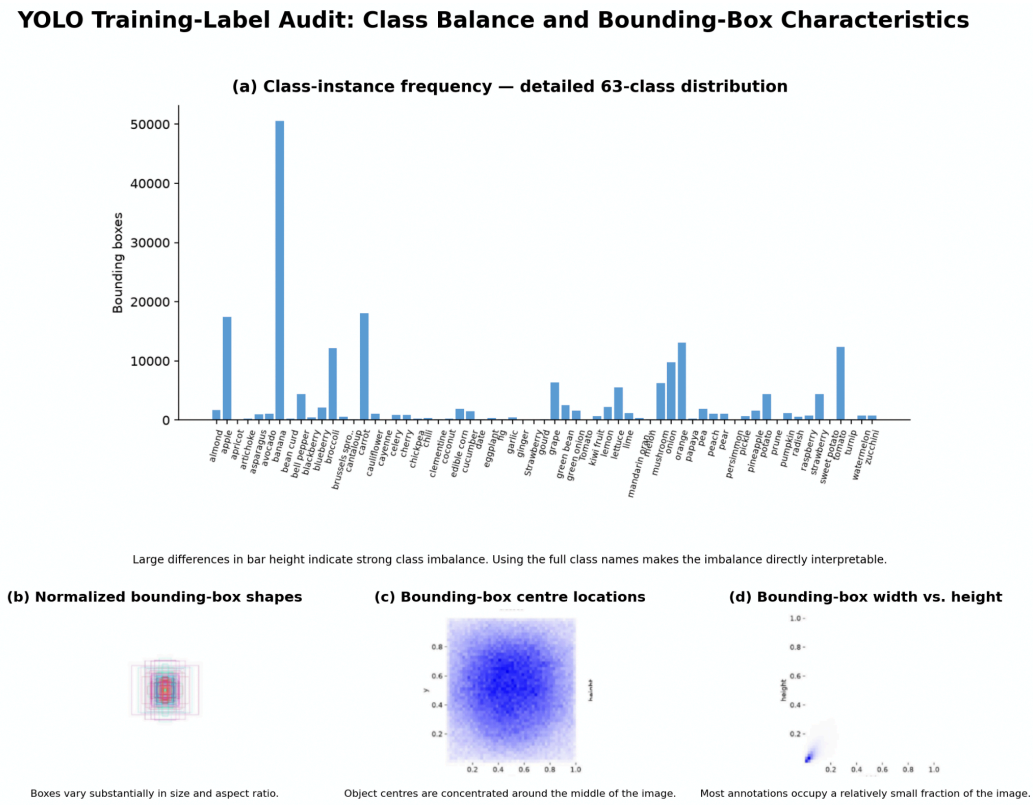
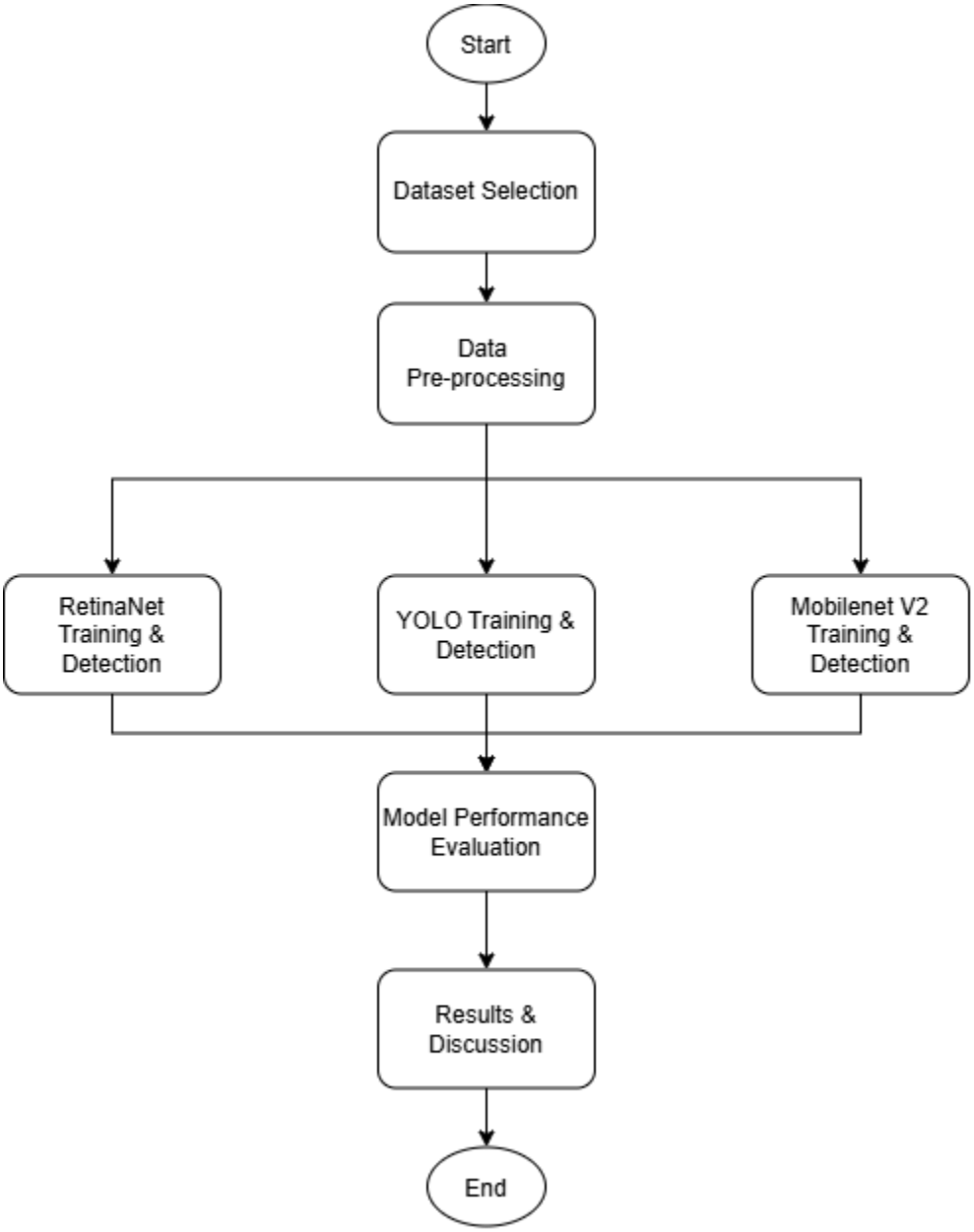


Figure 2.3: Ultralytics label audit showing the training-box class distribution

As shown in Figure 2.3, the Ultralytics label audit reveals notable class imbalance, with some classes containing thousands of bounding boxes and others far fewer, alongside considerable variation in bounding box positions, sizes, and aspect ratios. Given these characteristics this study evaluates Ultralytics YOLO11m on a 63-class fruit and vegetable dataset, providing further evidence of the model's effectiveness and limitations under conditions involving many categories, unequal class representation, varying object scales, visual similarity, and occlusion.

3. Methodology

3.1 System flowchart / activity diagram



3.2 Description and analysis of dataset

The dataset we used for our model training is the LVIS Fruits and Vegetables Dataset, obtained from Kaggle. It is a subset of the original Large Vocabulary Instance Segmentation (LVIS) dataset, which was developed for large-scale object detection and instance segmentation. The original LVIS dataset contains more than 1,000 object categories and follows a naturally long-tailed category distribution, where some object classes occur much more frequently than others (Gupta et al., 2019). The fruit and vegetable subset was created by selecting images containing relevant food categories and converting the original COCO-format annotations into YOLO-format annotations. The resulting dataset contains 8,221 images across 63 fruit and vegetable classes, consisting of 6,721 training images and 1,500 validation images. An additional 180 manually labelled test images are also provided. The selected classes include common fruits and vegetables such as apples, bananas, carrots, broccoli, oranges, potatoes, and watermelons.

Dataset Component (data.yaml)	Description
Number of classes	63
Training images	6721
Validation images	1500
Additional test images	180
Annotation format	YOLO .txt bounding-box format

The dataset uses the YOLO object-detection format. Each image has a matching text file that records the object class and the location of each bounding box. In this format, every object is written as class x_center y_center width height, with all coordinates scaled between 0 and 1. Before the dataset was released, the original COCO annotation files were converted from .json format into YOLO .txt files.

One important feature of the dataset is that it contains many different categories, but the categories are not evenly represented. For example, bananas, carrots, and apples appear much more often than some of the less common classes. This reflects the long-tailed distribution inherited from the original LVIS dataset. As a result, a model may learn to recognise common categories more easily, while producing weaker results for categories with fewer training examples.

The dataset also includes a wide range of visual appearances across its 63 categories. Some fruits and vegetables look similar because they have comparable colours, shapes, or textures. In addition, objects may appear in different positions, sizes, orientations, and surroundings. These variations make the dataset useful for testing whether a model can both identify and locate different types of fruits and vegetables.

There are also some issues with the dataset labels. For example, it includes both Tomato/tomato and Strawberry/strawberry as separate category names. These duplicate labels were inherited from the original LVIS class definitions. The capitalised versions appear in only a small number of images, so they are unlikely to have a major effect on the overall results. However, they may still cause some confusion when analysing the performance of individual classes.

Before the dataset was used in this project, irrelevant images were removed, the required fruit and vegetable categories were selected, and the annotations were converted from COCO format to YOLO format. During training, the images were resized and normalised to match the requirements of each detection model. No manually designed visual features were needed because the deep learning models learned these features directly from the images. Overall, the dataset is suitable for comparing models on multi-class fruit and vegetable object detection.

3.3 Algorithm selection & description of algorithm(s)

3.3.1 RetinaNet

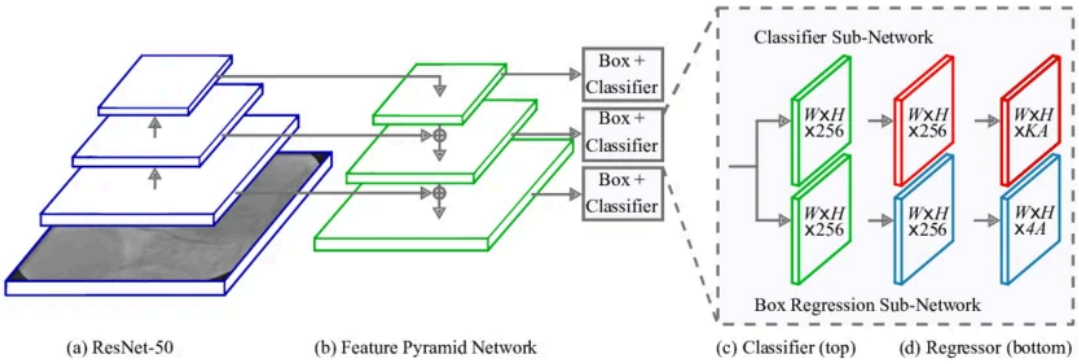


Figure 3.1: RetinaNet architecture with ResNet-50 and Feature Pyramid Network (FPN)

As shown in Figure 3.1, RetinaNet is composed of a ResNet-50 backbone, a Feature Pyramid Network (FPN), a classification subnet, and a box regression subnet. The ResNet-50 backbone extracts visual features from the input images, while the FPN generates feature representations at different scales to improve the detection of objects with varying sizes. The classification subnet is responsible for predicting the class of each object, while the box regression subnet predicts the coordinates of the corresponding bounding boxes.

In this project, RetinaNet with a ResNet-50 backbone and FPN is used to detect and classify 63 categories of fruits and vegetables. During training, each image is provided with its corresponding ground-truth bounding boxes and class labels. Data augmentation is applied to the training images to introduce greater variation in the dataset and help the model perform more effectively under different image conditions.

During training, RetinaNet learns the visual features and relationships required to identify objects and determine their locations within an image. As a one-stage object detection model, RetinaNet performs classification and bounding box regression within a single detection framework without relying on a separate region proposal stage. This allows the model to directly detect multiple fruits and vegetables within the same image.

After the training process, the trained RetinaNet model is used to generate predictions on the validation dataset. For each detected object, the model produces a predicted class, a confidence score, and a bounding box. These predictions are compared with the ground-truth annotations to determine whether the detected objects and their locations are correct.

The performance of RetinaNet is evaluated using several metrics, including precision, recall, F1-score, and Intersection over Union (IoU). Precision and recall measure the model's ability to correctly identify objects, while F1-score provides a balance between these two measures. IoU is used to evaluate the overlap between the predicted and ground-truth bounding boxes. The evaluation results are then used to assess the effectiveness of RetinaNet and compare it with the other object detection algorithms implemented in this project.

3.3.2 MobileNet V2

In this project, MobileNetV2 is used as the feature extraction backbone of a lightweight SSDLite object detection model. The model receives a complete RGB image and performs both fruit and vegetable classification and object localisation. ImageNet-pretrained MobileNetV2 weights are used to initialise the backbone, allowing previously learned visual features to be adapted to the 63 fruit and vegetable categories in the dataset.

To support the detection of objects at different scales, feature maps are extracted from different stages of the MobileNetV2 backbone. Additional depthwise separable convolution layers are used to generate lower-resolution feature maps, which are then passed to the SSDLite detection head for object classification and bounding-box prediction. In the baseline configuration, the model uses an input resolution of 320 × 320 pixels and produces six multi-scale feature maps for detection.

A two-phase transfer-learning strategy is used during training. In the first phase, the pretrained MobileNetV2 backbone is frozen while the additional detection layers are trained. In the second phase, selected later stages of the backbone are unfrozen and fine-tuned using a lower learning rate. The pretrained Batch Normalisation statistics are kept frozen during training to maintain stable feature representations when using a relatively small batch size. The model is trained using both classification loss and bounding-box regression loss.

Four MobileNetV2 configurations were evaluated in this project. The experiments progressively introduced small-object anchors, higher input resolution, high-resolution feature extraction, additional backbone fine-tuning, and object-crop augmentation. The final optimized model uses a 512 × 512 input resolution, a MobileNetV2-SSDLite HighRes architecture, and a small-object high-resolution anchor configuration. It also applies additional backbone fine-tuning and object-crop augmentation to improve the detection of small and difficult objects.

During inference, confidence filtering and Non-Maximum Suppression (NMS) are applied to remove low-confidence and highly overlapping predictions. The optimized model uses a lower confidence threshold than the baseline to retain more candidate detections and improve detection sensitivity. For each accepted prediction, the model produces a predicted class, confidence score, and bounding box.

3.3.3 YOLO

YOLO11m was selected as the main YOLO architecture for this study because it provides a balance between detection performance, computational cost and inference speed. Its multi-scale feature extraction lets the model localise and classify fruits and vegetables of widely different sizes in a single forward pass, without separate stages for detection and classification.

The model scale itself was held constant throughout the focal-loss refinement experiment. All experiments were conducted using one RTX 4060 laptop GPU, with a target inference latency below 12 ms/image as a threshold representative for typical YOLO applications such as fruit detection on a regular-speed conveyor belt. At this fixed scale, YOLO11m contains 20.1 million parameters and requires 68.5 GFLOPs. Therefore, the main experimental variable was the classification loss algorithm, while the standard YOLO11m architecture, detection head, training data and inference path all kept identical to the baseline.

YOLO11m training was initialised from COCO-pretrained weights and fine-tuned at 640 × 640 resolution using AdamW optimiser. The initial stage used a learning rate of 0.001, while the refinement stage used 0.0001, with a batch size of of 12, weight decay of 0.0005, seed fixed at 0, and an early-stopping patience of 300 epochs against a ceiling of 600 which are sized for an overnight training budget. The training was interrupted 134 epochs, with the best baseline checkpoint obtained at epoch 91 and subsequently used for refinement.

The dataset's strong class imbalance motivated a second design choice: replacing the standard binary cross-entropy classification loss with a per-class inverse-frequency focal loss for the refinement stage:

$$L_{cls}(c) = -alpha_c(1 - p_{t,c})^{gamma}\, BCE(p_c, y_c)$$

Where γ(gamma) = 1.5, and each class weight $\alpha_{(c)}$ is calculated from inverse class frequency using an exponent of 0.5 and normalised across all 63 classes. The focal term suppresses the influence of easy examples, while the class weights increase the contribution of rare classes. Thus, the experiment isolates the effect of class-balanced focal loss while keeping the model architecture and evaluation data constant.

3.4 Evaluation metrics

To evaluate the performance of the object detection models, four main evaluation metrics are used in this project: Object Accuracy, Precision Accuracy, Classification Accuracy, and Mean Intersection over Union (Mean IoU). These metrics evaluate different aspects of the detection process, including the ability to detect existing objects, the reliability of predicted detections, the correctness of object classification, and the accuracy of object localisation.

Before calculating these metrics, the predicted bounding boxes are matched with the corresponding ground-truth bounding boxes. A predicted object is considered successfully matched when the Intersection over Union (IoU) is equal to or greater than 0.5. Each ground-truth object can only be matched with one prediction to avoid duplicate detections.

3.4.1 Object accuracy

Object Accuracy measures the proportion of ground-truth objects that are successfully located by the model, regardless of whether the predicted class label is correct. A ground-truth object is considered successfully located when at least one predicted bounding box overlaps with it by an IoU of at least 0.5.

Formula:

$$\text{Object Accuracy} = \frac{\text{Number of Detected Objects}}{\text{Total Number of Ground Truth Objects}} \times 100\%$$

where:

1. Number of Detected Objects represent the number of ground-truth objects successfully matched with a predicted bounding box at the required IoU threshold.
2. Total number of Ground Truth Objects is the total number of actual annotated objects in the evaluation dataset.

A higher Object Accuracy indicates that the model is more capable of identifying and locating the presence of objects in an image. This metric focuses primarily on object localisation and detection capability, rather than classification correctness.

3.4.2 Classification Accuracy

Classification Accuracy evaluates how accurately the model assigns the correct fruit or vegetable category to objects that have already been successfully localised.
Predicted and ground-truth objects are first matched according to their bounding-box overlap. For matched objects with an IoU of at least 0.5, the predicted class label is then compared with the actual ground-truth class.

Formula:

$$\text{Classification Accuracy} = \frac{\text{Number of Correctly Classified Matched Objects}}{\text{Total Number of Matched Objects}} \times 100\%$$

A higher Classification Accuracy indicates that the model is more effective at distinguishing between the different fruit and vegetable categories after successfully locating the objects.

3.4.3 Precision

Precision measures how reliable the model's detections are. It represents the proportion of predicted objects that are correctly detected and classified.
A prediction is considered a True Positive (TP) when the predicted bounding box has an IoU of at least 0.5 with a ground-truth bounding box and the predicted class is correct. A False Positive (FP) occurs when a prediction does not correctly match a ground-truth object or predicts the wrong class.

Formula:

$$\text{Precision} = \frac{TP}{TP + FP} \times 100\%$$

where:

1. TP is the number of correctly detected and correctly classified objects.
2. FP is the number of incorrect detections.

A high Precision value indicates that most objects predicted by the model are valid detections, while a low Precision value indicates that the model produces a larger number of incorrect or false detections.

3.4.4 Recall

Recall measures the model's ability to identify the actual objects that exist in the images. It represents the proportion of ground-truth objects that are successfully detected with the correct class label.

Formula:

$$\text{Recall} = \frac{TP}{TP + FN} \times 100\%$$

where:

1. TP is the number of correctly detected and correctly classified objects.
2. FN is the number of ground-truth objects that were not correctly detected by the model.

A False Negative (FN) may occur when the model completely misses an object, when the predicted bounding box does not satisfy the required IoU threshold, or when the object is not correctly recognised according to the evaluation criteria.

A higher Recall value indicates that the model is capable of detecting a larger proportion of the actual fruits and vegetables present in the images.

3.4.5 Mean Intersection over Union (Mean IoU)

Intersection over Union (IoU) measures the degree of overlap between a predicted bounding box and its corresponding ground-truth bounding box. It is used to evaluate the localisation accuracy of the object detection model.

Formula:

$$\text{IoU} = \frac{\text{Area of Intersection}}{\text{Area of Union}}$$

where:
Area of Union is the total sum of Area of Predicted Bounding Box with Area of Ground Truth Bounding Box minus the Area of Intersection

The Mean IoU is calculated by averaging the IoU values of all successfully matched detections:

$$\text{Mean IoU} = \frac{\text{Sum of IoU Values of All Matched Objects}}{\text{Total Number of Matched Objects}}$$

A higher Mean IoU indicates that the predicted bounding boxes are more closely aligned with the actual ground-truth bounding boxes.

3.4.6 F1-score

The F1-score is a performance metric that combines Precision and Recall into a single value. It is particularly useful when both false detections and missed objects need to be considered. The F1-score is calculated using the harmonic mean of Precision and Recall.

Formula:

$$\text{F1-Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

A higher F1-score indicates that the model achieves a better balance between Precision and Recall. A low F1-score may indicate that the model either produces many incorrect detections or fails to detect many actual objects.

3.4.7 Mean Average Precision (mAP)

Average Precision (AP) summarizes the precision and recall curve for one class at a specific intersection over union threshold. mAP is calculated by averaging AP across all 63 fruit and vegetable classes. It measures both classification and localisation performance and is the main evaluation metric measured and reported by Ultralytics YOLO.

mAP50 is the mean AP calculated at IoU = 0.50. A prediction is considered correctly localised when its overlap with the ground-truth box is at least 0.50. This metric is relatively tolerant of small bounding-box errors.

$$mAP50 = (1 / C) \times \text{sum } AP_c \text{ at IoU } 0.50$$

mAP50-95 is the mean AP averaged over ten IoU thresholds from 0.50 to 0.95 in steps of 0.05. It is stricter because the higher thresholds require more accurate bounding boxes. A higher mAP50 or mAP50-95 indicates better overall object detection performance.

4. Result & Discussion

4.1 Results

4.1.1 RetinaNet

The final RetinaNet model was evaluated on the validation set to measure its object detection performance. The evaluation used **precision**, **recall**, **F1-score**, **classification accuracy**, **mean IoU** and **detection rate at an IoU threshold of 0.5**. The validation set contained **33,695** ground-truth objects.

Evaluation Metric	**Result**
Precision	47.39%
Recall	30.80%
F1-score	37.33%
Classification Accuracy (Matched Detections)	86.96%
Mean IOU	0.7261
Detection Rate @ IOU 0.5	26.78%

Table 4.1: RetinaNet Performance on the Validation Set

The model achieved a **precision of 0.3012** and a **recall of 0.4001**. The **F1-score was 0.3437**, while the **classification accuracy reached 0.8028**. The **mean IoU was 0.6993**, showing that the predicted bounding boxes were generally **reasonably close to the ground-truth boxes** when the objects were detected.

The detection rate at IoU 0.5 was 0.3212. This means that a proportion of the ground-truth objects were **successfully detected and matched with predictions** under the selected evaluation conditions.

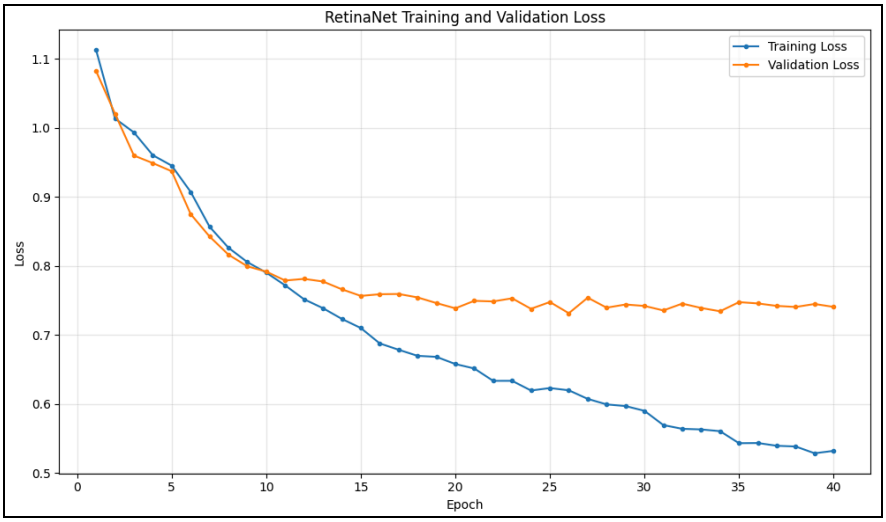


Figure 4.1: Graph of Training and Validation Loss in 40 epoch

Figure 4.1 also shows the training and validation loss during model training. The training loss generally decreased as the number of epochs increased, showing that the model was learning from the training data. The validation loss showed more variation during training, indicating that the model performance on unseen images was not always improving at the same rate as the training performance.

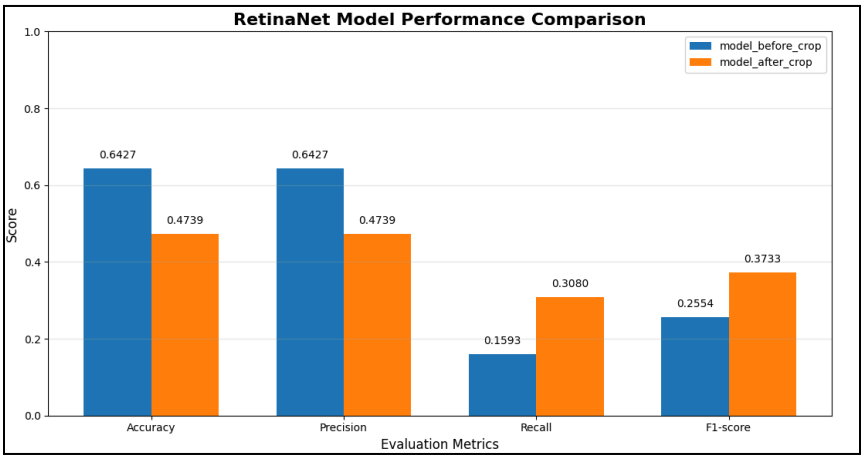


Figure 4.2: Training and Validation Loss over 40 Epochs between two model

Figure 4.2 compares the training and validation loss of the RetinaNet models before and after applying crop image augmentation over 40 epochs. The model with crop augmentation shows a different loss pattern compared to the original model. The crop images help the model focus more on smaller objects and different object areas during training. Overall, the graph shows how crop augmentation affects the model's training and validation performance.

4.1.2 MobileNet V2

The best model of the MobileNet V2 is evaluated using classification accuracy, mean IoU, precision, recall and the F1-score. In the training of MobileNet V2, there are four sets of the experiment of the config to control some specification detection of the model to test whether which experiment can produce a best model among the four settings.

Model	Classification Accuracy	Mean IoU	Precision	Recall	F1-score
Base	**89.76%**	**63.45%**	**19.25%**	**6.14%**	**9.31%**
Small object anchors	**87.67%**	**61.08%**	**14.82%**	**4.99%**	**7.47%**
Higher Resolution	**87.09%**	**64.57%**	**21.95%**	**10.87%**	**14.54%**
Optimized	**79.19%**	**67.31%**	**11.59%**	**34.06%**	**17.29%**

Table 4.2: MobileNetV2 model Performance

1. **Base Model:** using the standard MobileNetV2-SSDLite architecture with an input resolution of 320 × 320 pixels and the default anchor configuration. It is trained using the standard two-phase transfer learning strategy without additional small-object optimisation.

2. **Small object anchors model:** keeps the same 320 × 320 input resolution and standard MobileNetV2-SSDLite architecture as the baseline, but replaces the default anchors with a small-object anchor configuration to improve the detection of smaller fruits and vegetables.

3. **Higher Resolution model:** increases the input resolution from 320 × 320 to 512 × 512 pixels while using the standard MobileNetV2-SSDLite architecture together with the small-object anchor configuration. The higher resolution is intended to preserve more visual information for small objects.

4. **Optimized model:** uses a 512 × 512 input resolution with the optimized MobileNetV2-SSDLite HighRes architecture and a small-object high-resolution anchor configuration. In addition, both Stage 2 and Stage 3 of the MobileNetV2 backbone are fine-tuned, and a 50% object-crop augmentation probability is applied. The model also uses adjusted detection settings, including a lower positive IoU threshold, a higher negative-to-positive ratio, a lower confidence threshold, and a larger number of candidate detections.

Among the four experiments, the Optimized model achieved the best overall detection performance. It obtained the highest Recall of **34.06%**, the highest F1-score of **17.29%**, and the highest mean IoU of **67.31%**. In comparison, the Base model achieved the highest Classification Accuracy of **89.76%**, while the Higher Resolution model achieved the highest Precision of **21.95%**.

The results also show a progressive improvement in detection coverage across the later experiments. Recall increased from **6.14%** in the Base model to **10.87%** in the Higher Resolution model and further increased to **34.06%** in the Optimized model. Similarly, the F1-score increased from **9.31%** in the baseline to **14.54%** and finally **17.29%** in the Optimized Model. The Small-object Anchors Model achieved the lowest Recall and F1-score among the four configurations.

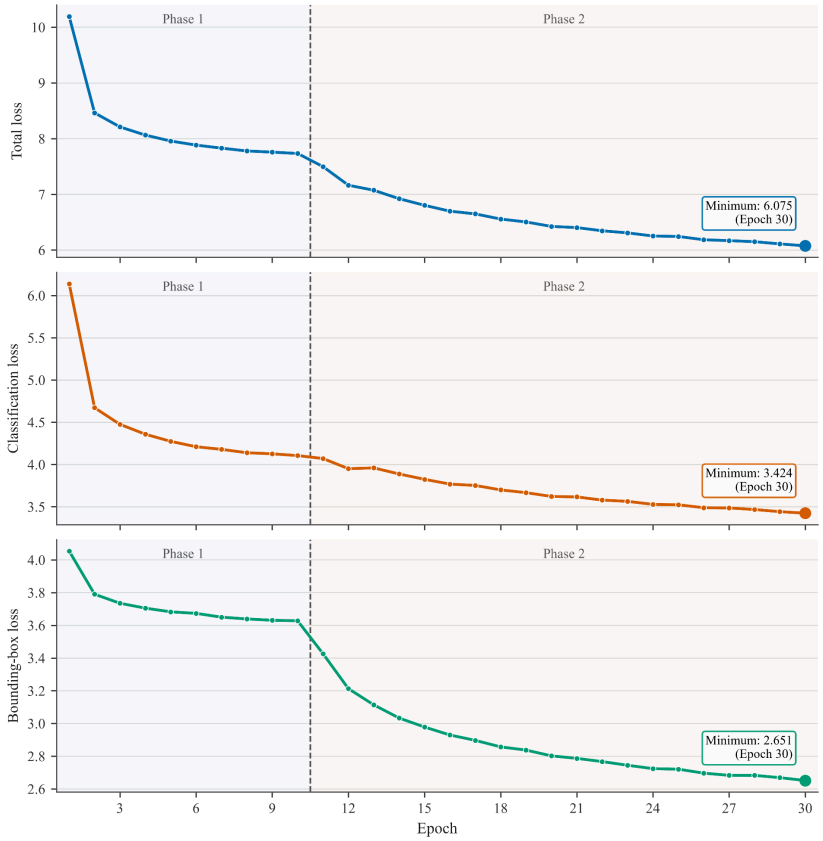


Figure 4.3: Training loss of the optimized MobileNetV2 model

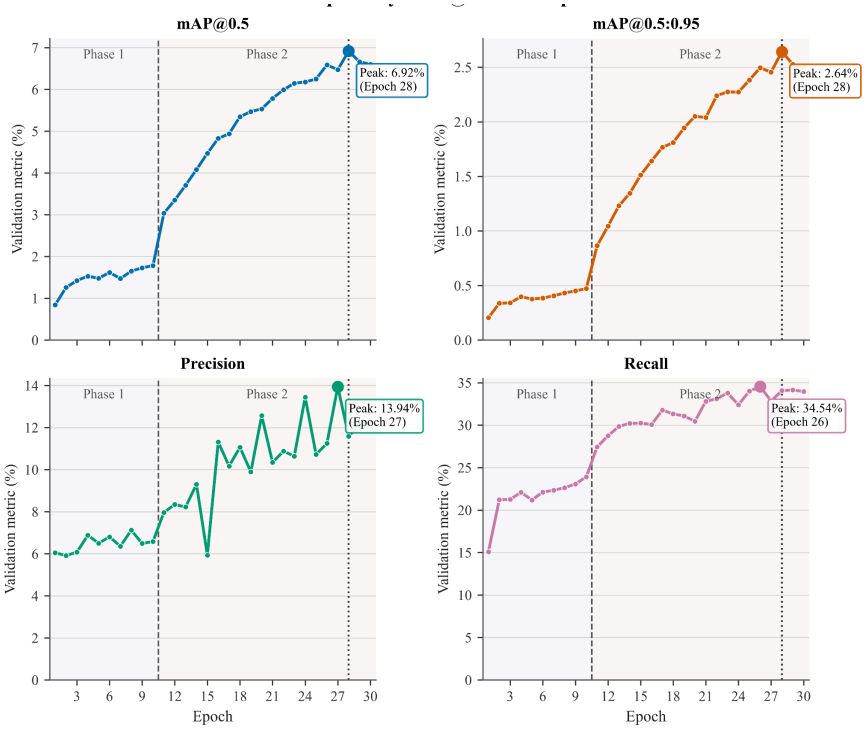


Figure 4.4: Validation Performance of optimized model, Best checkpoint by mAP@0.5:0.95 in Epoch 28

Figure 4.3 shows the training losses of the optimized model across the 30 training epochs. The total loss, classification loss, and bounding-box regression loss continuously decreased throughout training. The total loss decreased from approximately 10.2 in the first epoch to 6.08 at Epoch 30, while the classification and bounding-box losses decreased to approximately 3.42 and 2.65, respectively. The continued reduction in all three losses indicates that the model was progressively learning both object classification and localisation during training.

Figure 4.4 shows the validation performance of the optimized MobileNetV2-SSDLite model. During Phase 1, the improvement in validation performance was relatively gradual. A clearer improvement occurred after Phase 2 began, where additional MobileNetV2 backbone layers were fine-tuned. The mAP@0.5 increased to a maximum of **6.92%**, while **mAP@0.5:0.95** reached **2.64%** at Epoch 28. Recall also increased substantially during the second training phase and reached a maximum of **34.54%** at Epoch 26. Based on the highest **mAP@0.5:0.95**, Epoch 28 was selected as the best checkpoint for the optimized model.

4.1.3 YOLO

The focal-loss refinement was better overall than the baseline, but the change was a precision and recall trade-off rather than an improvement in every metric which means the model found more labelled objects but also produced more false positives. At epoch 16, focal YOLO11m reached **mAP50 0.3537 and mAP50-95 0.2166**, the relative gains are 4.4% and 6.3% respectively **compared with the baseline.**

The deployed best.pt checkpoint represents a slightly different balance. At epoch 4 it recorded the **highest mAP50-95 with 0.2173,** it is therefore the correct deployment checkpoint when mAP50-95 is the primary selection metric.

Both focal checkpoints **surpassed the original LVIS dataset author's trained YOLO** specialist-v3.pt model. At the epoch 16 operating point, focal YOLO11m is **higher by 43.3%** in mAP50, **18.6%** in mAP50-95, **12.3%** in precision, and **52.6%** in recall, while reducing latency by **51.9%**. To compare even further, the focal model is **70.3% smaller** on disk, with **70.5% fewer parameters** and **73.5% less compute** than the original author model. This is an improvement over the author model on every reported metric with substantial budget efficiency.

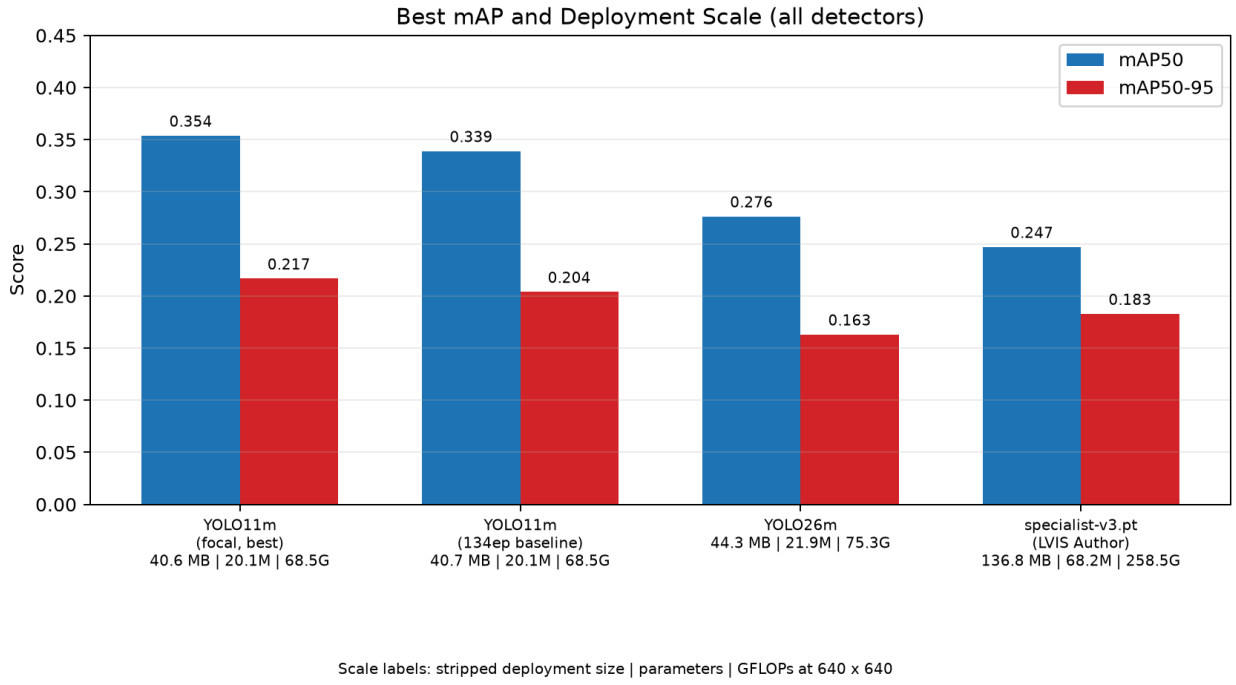


Figure 4.5: mAP50 and mAP50-95 comparison for all 4 models including the LVIS author's model

Model	Training	Epoch	mAP50	mAP50-95	Latency
YOLO11m focal	focal loss (gamma 1.5)	16	**0.3537**	0.2166	10.77ms
YOLO11m focal	Focal loss (gamma 1.5)	**4**	0.3456	**0.2173**	**9.40ms**
YOLO11m baseline	AdamW 1e-3 fine-tune	91	0.3389	0.2038	10.06ms
YOLO26m	MuSGD fine-tune	46	0.2762	0.1629	20.05ms
specialist-v3.pt	LVIS Author	35	0.2469	0.1827	22.39ms

Latency are subjective to the hardware performance

Table 4.4: Ultralytics metrics logged for all of the YOLO Models performance

Metrics for the selected best weight:

Metric	Epoch 4	Epoch 16
TP / FP / FN	1,790 / 3,161 / 935	1,740 / 3,015 / 985
Precision @ conf. 0.25	36.15%	36.59%
Recall @ conf. 0.25	65.69%	63.85%
F1 @ conf. 0.25	46.64%	46.52%
Classification Accuracy	72.40%	71.19%
Matched Mean IoU	79.11%	78.71%
Detection Rate @ IoU 0.5	85.50%	84.18%

Table 4.5: YOLO Performance on the Validation Set

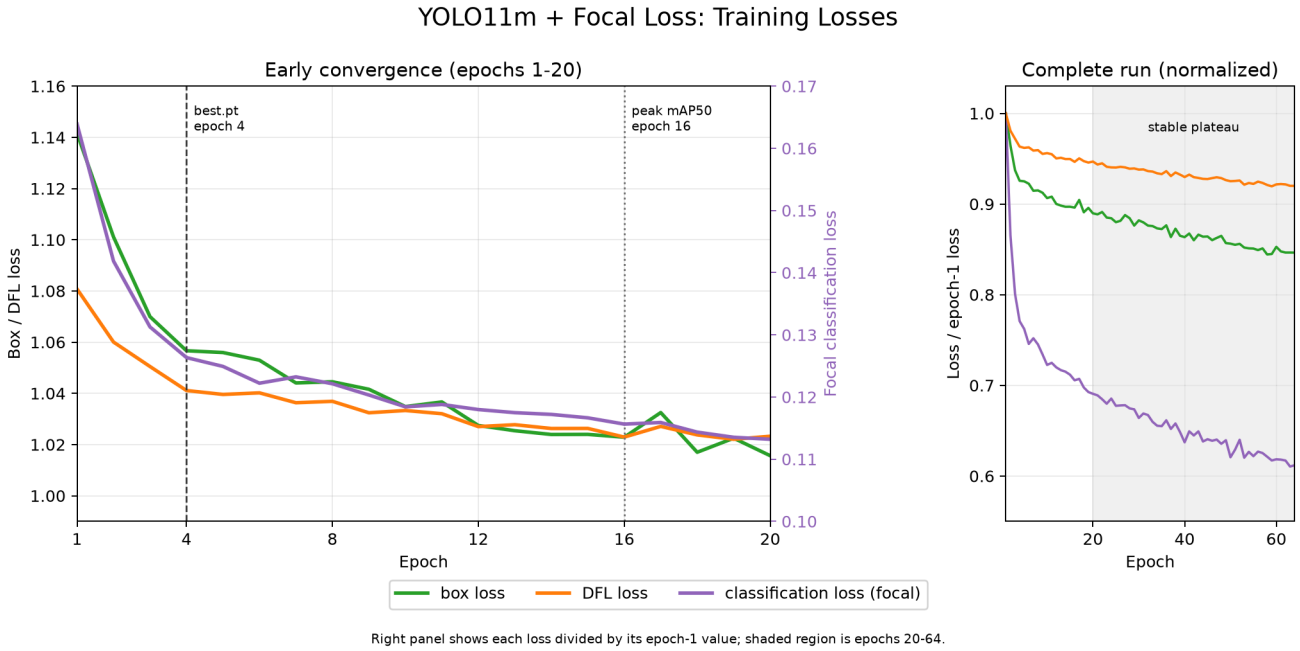


Figure 4.6: Focal-refinement training losses and checkpoint epochs

Figure 4.6 shows all three losses: box, DFL, and focal classification, they're falling sharply during the early refinement epochs before flattening around epoch 20, marking the shift into stable fine-tuning. Because the focal classification loss is scaled by the focal term and per-class weights, its lower magnitude isn't directly comparable to the box and DFL losses. Checkpoint selection at epochs 4 and 16 was based on validation performance rather than training loss: epoch 4 gave the highest mAP50-95 and was deployed as best.pt, while epoch 16 gave the highest mAP50. Training loss kept falling slightly beyond these points, but validation performance didn't follow, this illustrates why checkpoints should be chosen on validation metrics, not training loss alone.

4.2 Discussion/Interpretation

4.2.1 RetinaNet

The results show that the RetinaNet model was able to detect fruits and vegetables, but it still missed quite a lot of objects. The recall was 0.4001, while the precision was 0.3012. This means the model could find some of the objects in the images, but it also gave quite a number of incorrect detections. In other words, the model was more focused on finding possible objects, even though some of these predictions were not correct.

The confidence threshold also had an effect on the results. When the threshold is lower, the model keeps more predictions, so it has a better chance of finding more objects. However, this also means that more incorrect predictions are kept. This creates a trade-off between recall and precision. Therefore, the final result is affected not only by the trained model, but also by the confidence threshold used during testing.

The training and validation loss graphs also give some information about how the model learned. The training loss generally decreased during training, which shows that the model was learning from the training images. However, the validation loss was less stable and did not always decrease together with the training loss. This may mean that the model started to fit the training data too closely. As a result, further training did not always lead to better performance on unseen validation images.

Another major problem was the large number of small objects in the dataset. Many fruits and vegetables occupy only a small area of the image, so there are fewer pixels available for the model to learn their shape and position. Because of this, some small objects were missed, while some predictions only covered part of the object.

The crowded images also made detection more difficult. Some images contain several fruits or vegetables very close to each other, and some objects are partly hidden behind others. In these cases, the model may have trouble deciding where one object ends and another begins. Some predictions may also be removed during non-maximum suppression when two boxes overlap too much.

The mean IoU of 0.6993 shows that the bounding boxes were reasonably close to the ground-truth boxes for the objects that were detected. However, the detection rate was only 0.3212. This suggests that the main problem was not only the accuracy of the bounding boxes, but also the number of objects that the model failed to detect in the first place.

These results are still relevant to the main aim of the project, which is to develop a model that can detect and classify fruits and vegetables. The RetinaNet model was able to produce

correct detections, so it can be considered suitable for the task. However, the model still has problems with small objects, overlapping objects and partially hidden objects. These are important areas to improve because they occur quite often in the dataset.

Overall, the current model can handle the basic detection task, but its performance is still limited in more difficult images. Future improvements could include using higher-resolution feature maps to help with small objects, as well as testing different confidence and NMS thresholds. These changes may help the model detect more objects while also reducing incorrect predictions.

4.2.2 MobileNet V2

The experimental results show that the performance of MobileNetV2 was strongly affected by the configuration used during training. Among the four experiments, the optimized model achieved the best overall detection performance, with the highest Recall of 34.06%, F1-score of 17.29%, and Mean IoU of 67.31%. Compared with the baseline model, which achieved only 6.14% Recall, the optimized configuration detected a substantially larger proportion of the objects present in the validation images. This indicates that the modifications introduced in the final experiment mainly improved the model's ability to locate objects that were previously missed.

The comparison between the experiments also shows that changing only the anchor configuration was not sufficient to improve the model. The small-object anchor experiment produced lower Precision, Recall, F1-score, and Mean IoU than the baseline. In contrast, when the input resolution was increased from 320 × 320 to 512 × 512 pixels, the Precision increased to 21.95% and the F1-score increased to 14.54%. This suggests that preserving more spatial information was more beneficial than modifying the anchor configuration alone. This is particularly relevant to the dataset because many annotated fruits and vegetables occupy relatively small regions of the images.

The optimized model further improved detection coverage by combining higher input resolution, a high-resolution detection architecture, small-object high-resolution anchors, additional backbone fine-tuning, and object-crop augmentation. Recall increased from 10.87% in the higher-resolution experiment to 34.06% in the optimized experiment, while Mean IoU increased from 64.57% to 67.31%. These results show that the final configuration was able to detect more ground-truth objects while also slightly improving the localisation quality of successfully matched detections.

The training and validation curves provide further evidence that the relatively low absolute detection performance was not simply caused by a failure of the model to learn. The total loss, classification loss, and bounding-box regression loss continuously decreased across the 30 training epochs. Validation performance also improved more clearly after Phase 2 began, when additional MobileNetV2 backbone layers were fine-tuned. The mAP@0.5 increased to 6.92%, while mAP@0.5:0.95 reached 2.64% at Epoch 28. This indicates that the second training phase contributed positively to detection performance, although the absolute mAP values remained relatively low.

However, the improved Recall involved a clear trade-off in prediction reliability. The optimized model achieved a lower Precision of 11.59% and a lower Classification Accuracy of 79.19% than the other configurations. This was partly influenced by the more permissive training and inference settings used in the optimized experiment. The confidence threshold was reduced to retain more low-confidence predictions, while the positive anchor-matching threshold was also

reduced so that more candidate anchors could be treated as positive training examples. A larger candidate pool was also retained during inference. These settings increased the opportunity to recover small and difficult objects, but they also increased the likelihood of false-positive predictions. Therefore, the reduction in Precision should not be attributed only to the lightweight design of MobileNetV2.

The decrease in Classification Accuracy may also indicate that the optimized model detected a larger number of difficult objects that were harder to classify correctly. In addition, the dataset contains 63 fruit and vegetable categories with substantial class imbalance and several visually similar classes. These characteristics increase the difficulty of the detection task. Therefore, the final performance of MobileNetV2 should be interpreted as the combined effect of its lightweight architecture, dataset complexity, anchor and target-matching strategy, and the selected inference configuration.

4.2.3 YOLO

The training results demonstrate why imbalance handling was necessary. The baseline YOLO11m produced conservative predictions, the standard precisions were high, but recall was lower, indicating that many objects were missed. The refinement combined balanced image sampling with per-class inverse-frequency focal loss. Balanced sampling increased exposure to images that contained rare categories, while the focal term reduced the influence of easy predictions and emphasized difficult examples.

The largest loss reduction occurred during the early epochs, followed by a stable late-stage decline. Epoch 4 was selected for deployment because it achieved the highest mAP50-95, while epoch 16 achieved the highest mAP50. This distinction shows that continued training may improve performance at IoU 0.50 without improving every stricter IoU threshold. The fixed confidence results also favoured epoch 4 slightly in recall, F1, classification accuracy, mean IoU and detection rate.

Compared with the baseline, class-balanced focal refinement increased mAP50 from 0.3389 to 0.3537 and mAP50-95 from 0.2038 to 0.2166. The refinement increased object coverage but also retained more incorrect detections, creating a practical trade off between precision and recall. The GUI confidence slider also allows users to adjust this operating point to pragmatically adapt to various use cases for weighing over either sensitivity or fewer false positives, hence it should not be confused with the fixed Ultralytics operating-point benchmark on table 4.3.

The reason why 3 YOLO models (Focal YOLO11m, YOLO11m baseline and YOLO26m) were trained was to serve different experimental purposes and provide broader training experience across several object-detection approaches. The baseline YOLO11m established reference performance, while focal refinement examined the effect of imbalance handling. YOLO26m tested whether a newer, larger architecture improved performance, but the reality shows that the model does not translate into better generalisation on this dataset.

In comparison, RetinaNet achieved reasonable classification and localisation, but still missed many objects, while MobileNetV2-SSDLite reduced computational cost at the expense of recall and F1-score. YOLO11m showed a better balance between detection coverage and efficiency, with stronger overall performance while maintaining relatively low latency and model size.

5. Conclusion

5.1 Achievements

5.1.1 RetinaNet

This project successfully developed a RetinaNet model for fruit and vegetable detection using the LVIS Fruits & Vegetables dataset. The model was able to both identify the object class and locate the object with a bounding box, which meets the main goal of building an object detection system.

The final model achieved a precision of 0.3012, recall of 0.4001, F1-score of 0.3437, mean IoU of 0.6993, and detection rate of 0.3212 at IoU 0.5. The results show that the model was able to detect a reasonable number of objects, and the bounding boxes were generally close to the labelled objects when a detection was made.

Overall, the main objective was partially achieved. The RetinaNet model works and can produce useful detections, but the results are not yet strong enough to say that the model can reliably detect every object in the images. The project also helped identify the main problems affecting the model, especially small objects, overlapping objects and false positive detections.

5.1.2 MobileNet V2

This project successfully developed a MobileNetV2-based object detection model for fruit and vegetable detection using the LVIS Fruits & Vegetables dataset. The MobileNetV2 backbone was integrated with a lightweight detection head to perform both object classification and bounding-box localisation across the 63 fruit and vegetable categories. The completed model was able to process a full input image and produce multiple detected objects with their predicted classes, confidence scores, and bounding boxes.

Another achievement was the progressive improvement of the MobileNetV2 model through four experimental configurations. The baseline model achieved a Recall of **6.14%**, F1-score of **9.31%**, and Mean IoU of **63.45%**. After modifying the input resolution, anchor configuration, backbone fine-tuning strategy, and data augmentation settings, the final optimized model increased the Recall to **34.06%** and the F1-score to **17.29%**, while achieving the highest Mean IoU of **67.31%** among the four experiments. This shows that the optimized configuration was able to detect a substantially larger proportion of the ground-truth objects while maintaining improved bounding-box localisation.

A further achievement was identifying how different training configurations affected the detection performance of MobileNetV2. The small-object anchor configuration alone did not improve the baseline model, while increasing the input resolution produced better Precision and F1-score. The final optimized model further improved detection coverage through the combined use of higher-resolution input, additional backbone fine-tuning, modified anchor settings, and object-crop augmentation. Although the final model still showed a trade-off between Recall and Precision, the experiments demonstrated that the detection capability of a lightweight MobileNetV2 model could be significantly improved through systematic configuration and training adjustments.

5.1.3 YOLO

In this study, the YOLO component developed a real-time(<11ms) 63 class fruit and vegetable detector using the COCO-pretrained YOLO11m weights. One valuable thing I achieved and learned deeply was an experiment conducted to test whether a newer and larger architecture could improve detection performance, which is the YOLO26m model training. However, the results denied it with a lower mAP than the smaller 11m baseline model, this showed that increasing model and computational scale alone was not sufficient to improve performance on this dataset.

Another achievement was establishing the dataset author's specialist-v3.pt model as an external benchmark, which avoided the problem of simply "marking one's own homework" by relying on internally trained models. This provided a stronger reference for performance, model size and computational cost because the final YOLO11m could be evaluated against a model developed specifically for the same dataset. Surpassing this benchmark therefore provided stronger evidence that the refinement strategy produced meaningful improvements.

The third achievement was addressing the long-tailed class imbalance of the dataset, where common categories appeared far more frequently than rare classes (Figure 2.3, Chart A). Instead of accepting this limitation, the YOLO11m refinement introduced balanced image sampling and per-class inverse-frequency focal loss to increase the contribution of rare and difficult examples during training. This improved both mAP50 and mAP50-95 over the standard baseline and demonstrated how training strategy could compensate for dataset imbalance without changing the underlying YOLO11m architecture.

5.2 Limitations and Future Works

5.2.1 RetinaNet

The main limitation of the model is its difficulty in detecting small objects. Many fruits and vegetables in the dataset are relatively small, providing less visual information for the model to identify their locations accurately. As a result, some small objects may be missed, or the predicted bounding boxes may not cover the entire objects accurately.

Another challenge is the presence of crowded images. Several objects may appear very close to each other or partially overlap with one another. In such cases, the model may have difficulty distinguishing between individual objects. In addition, when predicted bounding boxes have a high degree of overlap, Non-Maximum Suppression (NMS) may remove some detections, which can affect the overall detection performance.

The model also showed relatively low precision compared with recall. This indicates that although the model was able to detect a certain number of the ground-truth objects, it also produced a considerable number of false positive detections. The selected confidence threshold can affect this balance between precision and recall, meaning that the current settings may not provide the optimal trade-off between detecting more objects and reducing incorrect detections.

For future work, the model could be improved by using higher-resolution feature maps, particularly to improve the detection of small objects. The confidence threshold and NMS settings could also be further investigated to achieve a better balance between precision and recall. In addition, more suitable data augmentation techniques could be introduced for images containing small, overlapping, or partially occluded objects.

Another possible improvement is to conduct a more detailed error analysis for each fruit and vegetable class. This could help identify which classes contribute most to the detection errors and allow future training to focus more on these challenging cases. With these improvements, the model could become more reliable and better suited for real-world object detection tasks.

5.2.2 MobileNet V2

One major limitation of the MobileNetV2-based detector is the trade-off between its lightweight architecture and its feature extraction capability. MobileNetV2 is designed to reduce computational complexity and model size, but this also limits the amount of visual information that can be represented by the network. This becomes more challenging in the current project because the model needs to distinguish between 63 fruit and vegetable categories, including several classes with similar colours, shapes, and textures. Although the model is able to classify successfully detected objects effectively, its lightweight backbone may still limit its ability to learn sufficiently detailed features for more difficult detection cases.

For future improvement, additional backbone layers could be fine-tuned while maintaining a lower learning rate to allow the pretrained MobileNetV2 features to adapt more closely to the fruit and vegetable dataset. Lightweight feature-enhancement techniques, such as improved multi-scale feature fusion or attention mechanisms, could also be investigated. Another possible approach is knowledge distillation, where a larger and more powerful object detector is used as a teacher model to transfer richer feature representations to MobileNetV2 while preserving its relatively lightweight structure.

Another important limitation is the long-tailed class distribution of the dataset. Some fruit and vegetable categories contain substantially more training examples than others, which can cause the model to focus more heavily on frequently occurring classes while learning weaker representations for rare categories. This is particularly important for MobileNetV2 because its limited model capacity must be shared across all 63 classes.

Future work could address this issue by introducing class-balanced training techniques. For example, weighted classification loss or class-balanced focal loss could assign greater importance to underrepresented categories during training. Oversampling and targeted data augmentation could also be applied to rare classes to increase their effective training samples. These approaches may improve the model's ability to recognise less frequently represented fruit and vegetable categories without significantly increasing the computational complexity of the MobileNetV2 architecture.

5.2.3 YOLO

The primary limitation is the long-tailed class distribution. Although balanced sampling increases the exposure of rare classes during training, repeatedly sampling the same images may still lead to overfitting. Future work should therefore collect additional images for underrepresented categories and evaluate per-class AP to determine which classes benefit most from the refinement.

The dataset also contains duplicate or inconsistent categories, such as Strawberry/strawberry and Tomato/tomato. These inconsistencies may cause classification confusion even when an object is correctly localised. Future data preparation should merge duplicate labels, review incorrect annotations and use confusion matrices to identify visually similar or frequently confused classes.

Small objects, occlusion and crowded scenes also remain challenging, particularly when the available images contain limited visual detail. While YOLO already performs multi-scale feature extraction, multi-scale training was not applied in this study. Future experiments could therefore evaluate multi-scale training, targeted small-object augmentation and lightweight feature-fusion methods while monitoring their effects on memory usage and inference latency. These trade-offs are especially important when considering deployment, where improving detection performance must be balanced against hardware constraints.

The stripped checkpoint is 40.6 MB, but practical runtime memory also includes model activations and application buffers. For deployment on low-RAM embedded or industrial systems, future work could explore FP16 or INT8 quantisation, structured pruning, knowledge distillation and deployment through TensorRT or ONNX Runtime. INT8 conversion may reduce storage further, but its effects on accuracy, latency and memory usage would need to be evaluated again after conversion.

Another limitation is that the current model detects only the 63 fruit and vegetable categories defined by the dataset. Characteristics such as ripeness, freshness and defects are not included in the available labels and would require a separately annotated dataset and new evaluation. In addition, a smaller model distilled from YOLO11m could be investigated for conveyor systems, automated sorting equipment and edge inspection devices where lower memory usage and latency are important.

Reference & Source

1. Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., & Chen, L.-C. (2018). MobileNetV2: Inverted residuals and linear bottlenecks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (pp. 4510–4520). https://doi.org/10.1109/CVPR.2018.00474

2. Zhou, Z., Song, Z., Fu, L., Gao, F., Li, R., & Cui, Y. (2020). Real-time kiwifruit detection in orchard using deep learning on Android smartphones for yield estimation. *Computers and Electronics in Agriculture, 179*, 105856. https://doi.org/10.1016/j.compag.2020.105856

3. Lin, T., Goyal, P., Girshick, R., He, K., & Dollar, P. (2017). Focal loss for dense object detection. https://openaccess.thecvf.com/content_iccv_2017/html/Lin_Focal_Loss_for_ICCV_2017_paper.html

4. Kirk, R., Cielniak, G., & Mangan, M. (2020). L*a*b*Fruits: A Rapid and Robust Outdoor Fruit Detection System Combining Bio-Inspired Features with One-Stage Deep Learning Networks. Sensors, 20(1), 275. https://doi.org/10.3390/s20010275

5. Kalantar, A., Edan, Y., Gur, A., & Klapp, I. (2020). A deep learning system for single and overall weight estimation of melons using unmanned aerial vehicle images. *Computers and Electronics in Agriculture*, *178*, 105748. https://doi.org/10.1016/j.compag.2020.105748

6. Edozie, E., Shuaibu, A. N., John, U. K., & Sadiq, B. O. (2025). Comprehensive review of recent developments in visual object detection based on deep learning. *Artificial Intelligence Review*, *58*(9). https://doi.org/10.1007/s10462-025-11284-w

7. Badgujar, C. M., Poulose, A., & Gan, H. (2024). Agricultural object detection with You Only Look Once (YOLO) algorithm: A bibliometric and systematic literature review. *Computers and Electronics in Agriculture, 223*, 109090. https://doi.org/10.1016/j.compag.2024.109090

8. Jocher, G., & Qiu, J. (2024). *Ultralytics YOLO11* (Version 11.0.0) [Computer software]. Ultralytics. https://docs.ultralytics.com/models/yolo11/

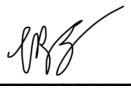
9. Pugazhendi, P., Badgujar, C. M., Sapkota, R., Dhillon, R., Rajesh, S., Sheela, J. J. J., & Ganapathy, M. R. (2026). Advances in agricultural fruit detection using You Only Look Once (YOLO) algorithm: A review. *Smart Agricultural Technology, 13*, 101896. https://doi.org/10.1016/j.atech.2026.101896

10. Zeng, T., Tong, J., Sun, X., Liu, J., He, X., Guan, Z., Liu, L., Jiang, N., & Wan, T. (2025). YOLO-FCAP: An improved lightweight object detection model based on YOLOv8n for citrus yield prediction in complex environments. *Smart Agricultural Technology, 12*, 101590. https://doi.org/10.1016/j.atech.2025.101590

Appendix A

Plagiarism Statement Form (student 1)

I, Name _____TANG RUI JIE_____Student ID_____26WMR12542_____ Programme ___RDS2S1_____Tutorial Group _____5_____confirm that the submitted work are all my own work and is in my own words.

Signature : 

Date : _____27/8/2026_____

Plagiarism Statement Form (student 2)

I, Name _CHONG ZHEN YANG_ Student ID _26WMR12484_ Programme _RDS2S1_ Tutorial Group _____5_____confirm that the submitted work are all my own work and is in my own words.

Signature : 

Date : _____27/8/2026_____

Plagiarism Statement Form (student 3)

I, Name (Block Capitals)_CHEW JIN YANG_Student ID_26WMR12479_Programme _RDS2S1_ Tutorial Group _____5_____confirm that the submitted work are all my own work and is in my own words.

Signature : *yang*

Date :_____27/8/2026_____

Appendix B

AI Usage Disclosure Form (student 1)

Student Name : Tang Rui Jie
Programme : RDS
Course : BMCS2074 ARTIFICIAL INTELLIGENCE
Assessment Title : Fruit and Vegetable Object Detection

1. AI Tool(s) Used (Check that all that apply)

- ☐ None (I did not use any AI tools for this assignment)
- ☑ ChatGPT (Version: 5.6)
- ☐ Deepseek
- ☐ Gemini
- ☑ Other: Claude

2. Nature of Assistance (Check all that apply)

- ☐ **Brainstorming:** Generating ideas, topics, or outlines.
- ☐ **Research:** Summarising long articles or finding concepts.
- ☑ **Editing:** Checking grammar, spelling, or sentence structure.
- ☑ **Coding/Math:** Debugging code or explaining a formula.
- ☐ **Creation:** Generating images, data, or draft text.

3. Disclosure Statement

Provide a brief explanation of how AI tools were utilised in completing the task and describe the measures taken to ensure that the final submission reflects your own independent work.

Example:

"I used ChatGPT to structure my initial ideas into an outline. Subsequently, I composed the full paragraphs independently and cross-checked the referenced dates against my textbook to ensure accuracy."

Your Statement:

Help me improve my model training speed and write the code for random crop images before training.

4. Verification & Integrity

- I have reviewed and verified the accuracy of all facts, data, and citations generated with the assistance of AI tools.
- I have properly acknowledged and cited any AI-generated text, ideas, or content in accordance with my instructor's requirements.
- I confirm that the reasoning, interpretation, and analysis presented in the final submission are entirely my own.

Student Signature : 

Date : 27/8/2026

Appendix B

AI Usage Disclosure Form (student 2)

Student Name : **Chong Zhen Yang**
Programme : **RDS2S1**
Course : **BMCS2074 ARTIFICIAL INTELLIGENCE**
Assessment Title : **Fruit and Vegetable Object Detection**

1. AI Tool(s) Used (Check that all that apply)

- ☐ None (I did not use any AI tools for this assignment)
- ☑ ChatGPT (Version: 5.6 sol)
- ☐ Deepseek
- ☐ Gemini
- ☐ Other: _____

2. Nature of Assistance (Check all that apply)

- ☐ **Brainstorming:** Generating ideas, topics, or outlines.
- ☐ **Research:** Summarising long articles or finding concepts.
- ☐ **Editing:** Checking grammar, spelling, or sentence structure.
- ☑ **Coding/Math:** Debugging code or explaining a formula.
- ☐ **Creation:** Generating images, data, or draft text.

3. Disclosure Statement

Provide a brief explanation of how AI tools were utilised in completing the task and describe the measures taken to ensure that the final submission reflects your own independent work.

Example:

"I used ChatGPT to structure my initial ideas into an outline. Subsequently, I composed the full paragraphs independently and cross-checked the referenced dates against my textbook to ensure accuracy."

Your Statement:

I used ChatGPT to debug my code. I complete the basics of the training model and create testing implementation and basic ui for the program by myself. I also used ChatGpt to structure my idea and make it into a list. Subsequently, I composed the paragraphs independently.

4. Verification & Integrity

- I have reviewed and verified the accuracy of all facts, data, and citations generated with the assistance of AI tools.
- I have properly acknowledged and cited any AI-generated text, ideas, or content in accordance with my instructor's requirements.
- I confirm that the reasoning, interpretation, and analysis presented in the final submission are entirely my own.

Student Signature : Yang

Date :**27/8/2026**

Appendix B

AI Usage Disclosure Form (student 3)

Student Name : **CHEW JIN YANG**
Programme : **RDS2S1**
Course : **BMCS2074 ARTIFICIAL INTELLIGENCE**
Assessment Title : **Fruit and Vegetable Object Detection**

1. AI Tool(s) Used (Check that all that apply)

- ☐ None (I did not use any AI tools for this assignment)
- ☐ ChatGPT (Version: _____)
- ☑ Deepseek
- ☐ Gemini
- ☐ Other: _____

2. Nature of Assistance (Check all that apply)

- ☐ **Brainstorming:** Generating ideas, topics, or outlines.
- ☑ **Research:** Summarising long articles or finding concepts.
- ☐ **Editing:** Checking grammar, spelling, or sentence structure.
- ☑ **Coding/Math:** Debugging code or explaining a formula.
- ☐ **Creation:** Generating images, data, or draft text.

3. Disclosure Statement

Provide a brief explanation of how AI tools were utilised in completing the task and describe the measures taken to ensure that the final submission reflects your own independent work.

Example:

"I used ChatGPT to structure my initial ideas into an outline. Subsequently, I composed the full paragraphs independently and cross-checked the referenced dates against my textbook to ensure accuracy."

Your Statement:

I used Deepseek to brainstorm what potential algorithms can be applied mathematically. I also used a RAG system (Notebooklm) to learn team members' codebase. All the final implementations, model training, scenario reasoning, analysis of results and decisions made presented in the submission are of my own work.

4. Verification & Integrity

- I have reviewed and verified the accuracy of all facts, data, and citations generated with the assistance of AI tools.
- I have properly acknowledged and cited any AI-generated text, ideas, or content in accordance with my instructor's requirements.
- I confirm that the reasoning, interpretation, and analysis presented in the final submission are entirely my own.

Student Signature : *yang*

Date : **27/8/2026**